

Normalización de la base de datos

Plataforma de Exámenes en Línea · PostgreSQL 16 · 22 tablas propias

Este documento explica cómo se llegó al modelo actual aplicando la primera, segunda y tercera forma normal (1FN, 2FN y 3FN). Todas las tablas y columnas mencionadas existen en `backend/apps/*/models.py`; el diagrama está en `docs/diagrama-er.png`.

1. Punto de partida: una hoja "plana"

Si el sistema se hubiera llevado en una sola hoja de cálculo, cada examen presentado por un estudiante sería una fila como esta:

Columna	Valor de ejemplo
examen	Parcial SQL
materia	Bases de Datos
docente	Ana Ruiz
correo_docente	ana@demo.com
estudiante	Luis Pérez
correo_estudiante	luis@demo.com
rol_estudiante	Estudiante
permisos_del_rol	exámenes.rendir, resultados.ver_propios
preguntas	¿Qué es una PK?, ¿Qué hace un INNER JOIN?
puntajes	1, 2
opciones_marcadas	PRIMARY KEY ; INNER JOIN
nota	4.2

Problemas de esta estructura:

- Hay celdas con varios valores (`permisos_del_rol`, `preguntas`, `puntajes`, `opciones_marcadas`).
- Los datos del docente, de la materia y del estudiante se repiten en cada fila.
- Cambiar el correo de un docente obliga a modificar muchas filas (anomalía de actualización).
- No se puede registrar un docente o una pregunta que aún no tenga exámenes (anomalía de inserción).
- Borrar el último examen de un estudiante borra también sus datos personales (anomalía de borrado).

2. Primera forma normal (1FN)

Regla: todos los valores son atómicos (una celda, un dato), no hay grupos repetidos y cada fila se identifica con una clave.

Qué se hizo:

- Se separó cada valor múltiple en su propia fila: una fila por cada **pregunta** que un estudiante responde en un examen.
- La clave natural queda como (**examen, estudiante, pregunta**).
- Los permisos de un rol dejaron de ser una lista dentro de una celda y pasaron a una relación propia (rol, permiso).

Resultado de la 1FN (una fila por pregunta respondida):

examen	estudiante	pregunta	puntaje_pregunta	opcion_marcada	nota
Parcial SQL	Luis Pérez	¿Qué es una PK?	1	PRIMARY KEY	4.2
Parcial SQL	Luis Pérez	¿Qué hace un INNER JOIN?	2	INNER JOIN	4.2

(Las columnas de docente, materia y datos del estudiante siguen repitiéndose en cada fila; eso se corrige en la 2FN y la 3FN.)

Dónde quedó en el proyecto:

Valor que era múltiple	Tabla que lo resuelve
Permisos de un rol	<code>rol_permisos</code> (<code>rol_id</code> , <code>permiso_id</code>)
Opciones de una pregunta	<code>opciones_respuesta</code> (una fila por opción)
Opciones marcadas en una respuesta	<code>respuesta_opciones</code> (una fila por opción elegida)
Archivos adjuntos	<code>archivos_adjuntos</code> (una fila por archivo)

3. Segunda forma normal (2FN)

Regla: estar en 1FN y que ningún atributo dependa solo de una **parte** de la clave compuesta.

Con la clave (**examen, estudiante, pregunta**) aparecen dependencias parciales:

Atributos	Dependen solo de	Problema
título, fechas, duración, estado, materia, docente	examen	Se repiten por cada estudiante y cada pregunta
nombre, correo, rol	estudiante	Se repiten por cada examen y cada pregunta
enunciado, tipo, dificultad	pregunta	Se repiten por cada estudiante y cada examen
orden, puntaje en el examen	(examen, pregunta)	No dependen del estudiante
estado de la asignación	(examen, estudiante)	No dependen de la pregunta
nota	(examen, estudiante)	No depende de la pregunta

Qué se hizo: cada grupo pasó a su propia tabla, y las relaciones muchos a muchos se resolvieron con tablas intermedias.

Dependencia	Tabla resultante	Clave
examen → sus datos	<code>exámenes</code>	<code>id</code>
estudiante → sus datos	<code>usuarios</code>	<code>id</code> (<code>email</code> es único)
pregunta → sus datos	<code>preguntas</code>	<code>id</code>
(examen, pregunta) → orden, puntaje	<code>examen_preguntas</code>	<code>id</code> + única (<code>examen_id</code> , <code>pregunta_id</code>)
(examen, estudiante) → estado	<code>asignaciones_examen</code>	<code>id</code> + única (<code>examen_id</code> , <code>estudiante_id</code>)
(asignación, número de intento) → fechas y estado	<code>intentos</code>	<code>id</code> + única (<code>asignacion_id</code> , <code>numero_intento</code>)
(intento, pregunta) → respuesta y puntaje	<code>respuestas_intento</code>	<code>id</code> + única (<code>intento_id</code> , <code>pregunta_id</code>)
intento → nota final	<code>resultados_examen</code>	<code>id</code> , relación 1 a 1 con <code>intentos</code>

Nota sobre intentos: el examen permite varios intentos (`exámenes.intentos_permitidos`). Por eso la nota y las respuestas dependen del **intento**, no solo del par (examen, estudiante). Esa es la razón de que exista la tabla **intentos** entre **asignaciones_examen** y **respuestas_intento**.

Nota sobre las claves: Django usa una clave sustituta (**id**) en todas las tablas. La clave natural compuesta se conserva como restricción **UNIQUE**, y es sobre esa clave natural que se hizo el análisis.

4. Tercera forma normal (3FN)

Regla: estar en 2FN y que ningún atributo no clave dependa de otro atributo no clave (no hay dependencias transitivas).

Dependencias transitivas encontradas y cómo se eliminaron:

Tabla donde estaba	Dependencia transitiva	Solución
exámenes	docente_id → nombre y correo del docente	Esos datos viven solo en usuarios ; exámenes guarda docente_id
exámenes	materia_id → nombre de la materia	Tabla materias ; exámenes guarda materia_id
usuarios	rol_id → nombre y descripción del rol	Tabla roles ; usuarios guarda rol_id
roles	rol → varios permisos	Tabla permisos + intermedia rol_permisos
preguntas	banco_id → título, materia y docente del banco	Tabla bancos_preguntas
respuestas_intento	opcion → texto de la opción	Tabla opciones_respuesta + intermedia respuesta_opciones
archivos_adjuntos	subido_por_id → datos del usuario	Se guarda solo la referencia a usuarios

Resultado: cada dato se guarda **una sola vez** y se obtiene mediante JOIN. Por ejemplo, el correo de un docente solo existe en **usuarios.email**; si cambia, se actualiza una fila y todos sus exámenes y bancos lo reflejan.

5. Modelo final por módulo

Módulo	Tablas	Qué guarda
Seguridad	usuarios , roles , permisos , rol_permisos , refresh_tokens	Quién es cada usuario, qué rol tiene y qué puede hacer
Banco de preguntas	materias , bancos_preguntas , preguntas , opciones_respuesta	El contenido evaluable
Exámenes	exámenes , examen_preguntas , asignaciones_examen	Qué preguntas lleva cada examen y a quién se asigna
Intentos	intentos , respuestas_intento , respuesta_opciones , resultados_examen , eventos_intento	Lo que ocurre cuando un estudiante presenta el examen
Complementarias	notificaciones , archivos_adjuntos	Avisos y archivos
Trazabilidad	auditoria , historial_cambios , historial_estado	Quién hizo qué y cuándo

Total: 22 tablas propias. (PostgreSQL muestra 29 porque Django agrega 7 internas: **django_migrations**, **django_content_type**, **django_session**, **django_admin_log**, **auth_permission**, **auth_group** y **auth_group_permissions**.)

Cardinalidades principales

Relación	Cardinalidad
Rol – Usuario	1 a N (un usuario tiene exactamente un rol)
Rol – Permiso	N a M (resuelta con <code>rol_permisos</code>)
Materia – Banco de preguntas	1 a N
Banco – Pregunta	1 a N
Pregunta – Opción de respuesta	1 a N
Examen – Pregunta	N a M (resuelta con <code>examen_preguntas</code>)
Examen – Estudiante	N a M (resuelta con <code>asignaciones_examen</code>)
Asignación – Intento	1 a N (hasta <code>intentos_permitidos</code>)
Intento – Respuesta	1 a N
Respuesta – Opción elegida	N a M (resuelta con <code>respuesta_opciones</code>)
Intento – Resultado	1 a 1

6. Reglas que la base de datos hace cumplir

No todo se valida en el código: la base también protege la integridad.

Restricción	Dónde	Qué garantiza
<code>UNIQUE</code> (email)	<code>usuarios</code>	No hay dos usuarios con el mismo correo
<code>UNIQUE</code> (rol, permiso)	<code>rol_permisos</code>	Un permiso no se asigna dos veces al mismo rol
<code>UNIQUE</code> (examen, pregunta)	<code>examen_preguntas</code>	Una pregunta no se repite en un examen
<code>UNIQUE</code> (examen, estudiante)	<code>asignaciones_examen</code>	Un examen no se asigna dos veces al mismo estudiante
<code>UNIQUE</code> (asignacion, numero_intento)	<code>intentos</code>	Los números de intento no se repiten
<code>UNIQUE</code> (asignacion) cuando estado = 'en_progreso'	<code>intentos</code>	Solo un intento en progreso por asignación
<code>UNIQUE</code> (intento, pregunta)	<code>respuestas_intento</code>	Una sola respuesta por pregunta en cada intento
<code>UNIQUE</code> (respuesta, opcion)	<code>respuesta_opciones</code>	Una opción no se marca dos veces
<code>CHECK</code> (fecha_fin > fecha_inicio)	<code>exámenes</code>	Las fechas del examen son coherentes
<code>CHECK</code> de un solo dueño	<code>archivos_adjuntos</code>	Un adjunto es de una pregunta o de un examen, nunca de ambos ni de ninguno
<code>on_delete=PROTECT</code>	claves hacia <code>usuarios</code> , <code>preguntas</code> , <code>materias</code>	No se puede borrar algo que ya tiene exámenes o resultados

7. Decisiones conscientes (desnormalización controlada)

El modelo cumple la 3FN, con las siguientes excepciones deliberadas. Se documentan porque son decisiones de diseño, no descuidos.

Dato	Por qué se guarda aunque se pueda calcular o repetir
<code>resultados_examen.puntaje_total</code> , <code>puntaje_maximo</code> , <code>nota_final</code>	Son valores derivados de las respuestas. Se guardan como registro oficial del resultado porque se actualizan cuando el docente califica una pregunta abierta, y así reportes, dashboard y constancias no recalculan la nota en cada consulta.
<code>respuestas_intento.es_correcta</code>	Resultado de la calificación automática, guardado para no recalcularlo. En preguntas abiertas queda en NULL hasta la revisión del docente.
<code>respuestas_intento.puntaje_obtenido</code>	En preguntas abiertas no se puede derivar: lo asigna el docente a mano.
<code>examen_preguntas.puntaje</code>	No es redundante con <code>preguntas.puntaje</code> : aquel es el valor por defecto ; este es el puntaje de la pregunta en ese examen , que depende de la pareja (examen, pregunta).
<code>intentos.fecha_limite</code>	Se calcula al iniciar el intento (inicio + duración, sin pasar de <code>fecha_fin</code> del examen). Se guarda porque la tarea programada de Celery filtra por esta columna cada 60 segundos para cerrar los intentos vencidos, sin recalcular el plazo de cada uno.
<code>exámenes.materia_id</code>	Se puede deducir de la materia del banco de cada pregunta del examen. Se conserva porque el examen necesita su materia desde que se crea, aún sin preguntas, y porque permite filtrar y listar exámenes por materia sin JOINS. La consistencia la garantiza el servicio <code>agregar_pregunta</code> , que rechaza preguntas de otra materia.
<code>entidad + entidad_id</code> en <code>auditoria</code> , <code>historial_cambios</code> , <code>historial_estado</code> y <code>notificaciones</code>	Referencia genérica (no es clave foránea): una misma tabla debe poder registrar cambios de cualquier entidad (examen, pregunta, intento, resultado). Una FK real obligaría a tener una tabla de auditoría por cada entidad.
<code>usuarios.nombre</code> en un solo campo	Se trata como un dato atómico para este sistema: no se filtra ni se ordena por apellido por separado.

8. Ejemplos de consultas que lo demuestran

Estas consultas se pueden ejecutar en pgAdmin para mostrar que los datos se reconstruyen con JOIN y no están duplicados.

Examen con su materia y su docente:

```
SELECT e.titulo, m.nombre AS materia, u.nombre AS docente, u.email
FROM examenes e
JOIN materias m ON m.id = e.materia_id
JOIN usuarios u ON u.id = e.docente_id;
```

Preguntas de un examen con su orden y puntaje:

```
SELECT ep.orden, p.enunciado, p.tipo, ep.puntaje
FROM examen_preguntas ep
JOIN preguntas p ON p.id = ep.pregunta_id
WHERE ep.examen_id = 1
ORDER BY ep.orden;
```

Permisos de cada rol (relación N a M):

```
SELECT r.nombre AS rol, p.nombre AS permiso
FROM roles r
JOIN rol_permisos rp ON rp.rol_id = r.id
JOIN permisos p ON p.id = rp.permiso_id
ORDER BY r.nombre, p.nombre;
```

Resultado de cada estudiante con su examen:

```
SELECT u.nombre AS estudiante, e.titulo AS examen, i.numero_intento, r.nota_final
FROM resultados_examen r
JOIN intentos i ON i.id = r.intento_id
JOIN asignaciones_examen a ON a.id = i.asignacion_id
JOIN examenes e ON e.id = a.examen_id
JOIN usuarios u ON u.id = a.estudiante_id
ORDER BY e.titulo, u.nombre;
```